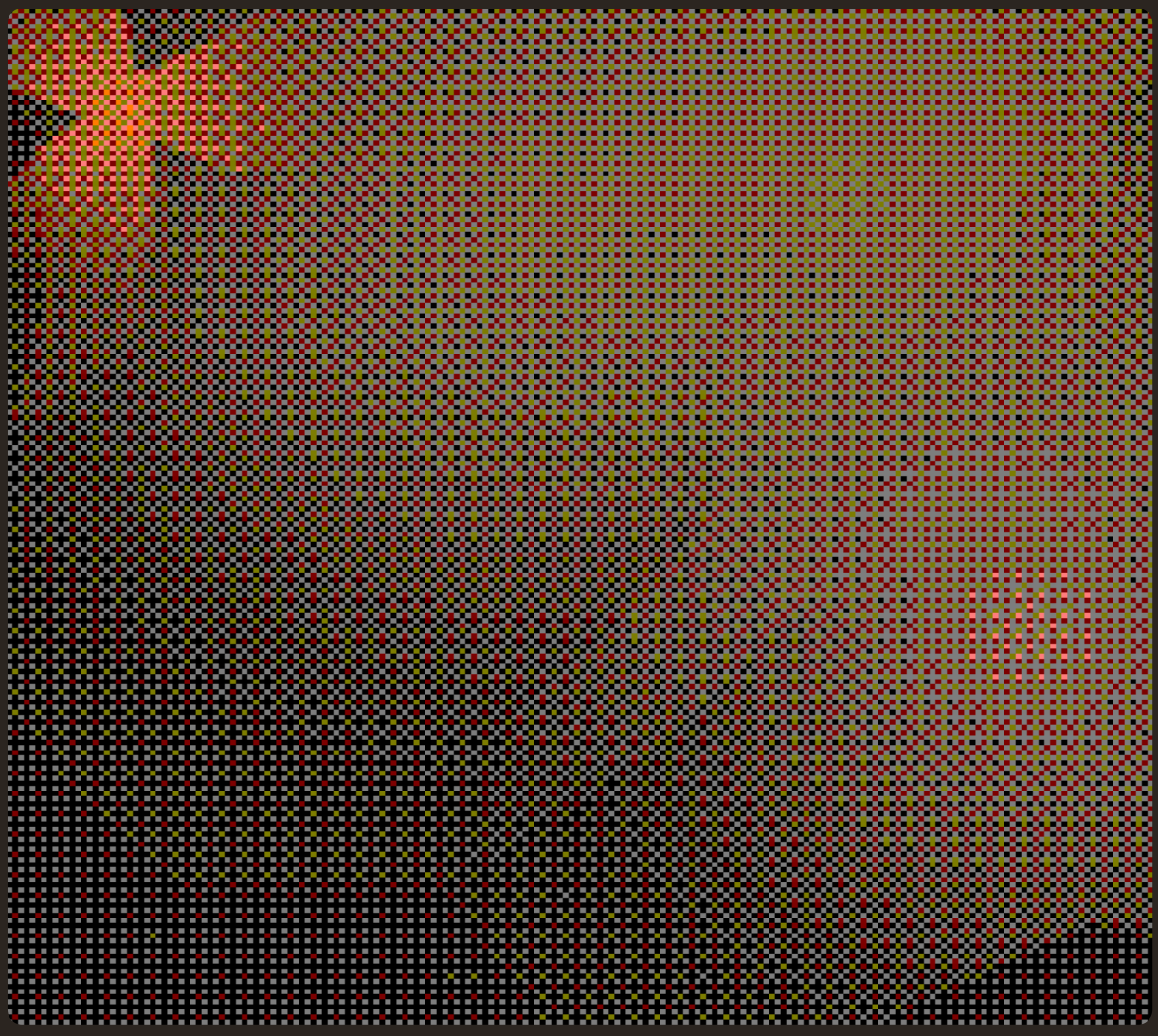
ZKSECURITY

Audit of Renegade's V2 Darkpool Circuits and Contracts

JANUARY 30TH, 2026 • TECHNICAL REPORT



Introduction

On January 19th 2026, zkSecurity started a security audit of Renegade's V2 darkpool implementation, covering both circuits and Solidity contracts. The code was shared via public GitHub repositories. The audit lasted two weeks with two consultants.

This is zkSecurity's fourth audit collaboration with Renegade. The first audit reviewed circuits and Stylus contracts in general, the second audit focused on atomic settlement, and the third audit covered the migration to Solidity contracts and malleable matches. A detailed description of the system can be found in those reports.

Scope

The Renegade team froze the following branches for this audit:

1. renegade/zksecurity-audit-1-19-26 at commit `3576ca43`
2. renegade-contracts/zksecurity-audit-1-19-25 at commit `2f8d4296`

The audit covered the V2 darkpool implementation:

- **Solidity contracts** (`renegade-contracts`). We reviewed the V2 darkpool Solidity implementation in `src/darkpool/v2` . The following new libraries in `src/libraries` were also in scope:
  - `merkle/` : The new Merkle mountain range implementation.
  - `ECDSA.sol` : Wraps the OpenZeppelin ECDSA implementation with helpers for the expected signature format.
- **Circuits** (`renegade`). We reviewed the V2 circuit implementation:
  - `circuits/circuit-types` : The types used throughout the circuits.
  - `circuits/circuits-core` : The circuit implementations.

Summary

The codebase demonstrates good overall quality, with well-structured code, thorough comments, and good test coverage in key areas across both the circuits and contracts. The architectural design of V2 (separating intents from balances and introducing proof linking between validity and settlement proofs) is well thought out and cleanly implemented.

We did not identify any critical or high-severity issues during this audit. We found two medium-severity issues: an unenforced constraint in the nullifier gadget that could allow `index=0` states to pass validation, and missing amount bitlength constraints on fee balance updates that could allow values to drift beyond the expected range. Both are soundness issues without immediate exploit paths but warrant attention.

We also identified one low-severity issue related to missing curve validation for user-provided signing authority keys, and several informational findings covering code hygiene items such as stale comments, dead code, and hardcoded developer paths.

We recommend addressing the medium-severity findings before deployment, as they represent deviations from the protocol's intended invariants. The Renegade team was responsive throughout the engagement and provided helpful context on design decisions.

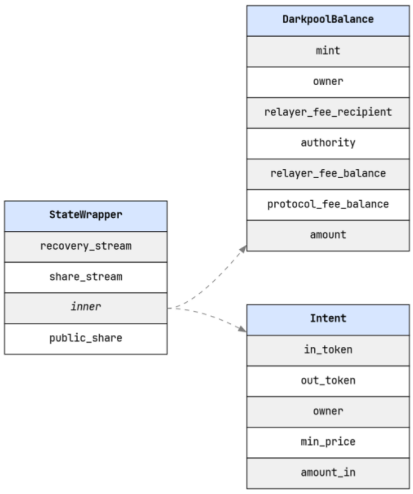
Overview

Protocol

Renegade is a dark pool trading platform that allows users to trade ERC-20 assets without revealing any information about their trades and balances. Version 2 of the protocol introduces a change from V1's wallet-centric model to an intent-and-balance-centric design. In V1, the core primitive was the wallet containing orders and balances, with circuits operating on wallet state transitions. V2 decouples these concepts: intents express trading preferences (token pair, amount, minimum price), while balances are

standalone state elements holding token amounts. This separation enables more flexible privacy configurations: users can have private intents with public balances, or fully private intents backed by private balances committed to the Merkle tree.

The protocol now follows a model reminiscent of the UTXO model. Briefly, balances and intents are committed to Merkle trees on-chain and every update references the old instances, invalidates them, and creates new ones. State wrappers encapsulate balances and intents:



In V2, a PRF is constructed from the Poseidon2 hash function with rate $r = 2$ and capacity $c = 1$, as:

$$PRF(\text{idx}; \text{secret\_seed}) \coloneqq Poseidon2([\text{secret\_seed}, \text{idx}])$$

There are two instances of such PRFs in `StateWrapper`, initialized with different seeds:

- `recovery_stream`: This instance is used to create identifier tags for on-chain state updates. Since this is a secure PRF, an external observer of the chain can only identify the tag if they possess the secret seed.
- `share_stream`: This instance is used to encrypt the balance or the intent via a stream cipher, with the resulting ciphertext denoted as `public_share`.

In the present state of the V2 implementation, no communication takes place between relayers for order matching. Thus, the MPC functionality which was characteristic of V1 has not been implemented yet. Instead, the matching is assumed to be performed internally for users interacting with the same relayer, with the relayer having access to the balances and intents in plaintext off-chain. However, a relayer cannot perform any token transfers beyond the scope of user-authorized intents: explicit authorization is required for deposits (Permit2 authorization), withdrawals (signature), intent creation (signature for creation, but not for fills), and cancellation (signature), with nonces and nullifiers preventing replay attacks.

The settlement layer of this second version supports multiple privacy configurations: private settlement (both parties' obligations hidden), public settlement (obligations visible but state updates private), and bounded settlement (trade size determined at runtime by an external matcher rather than fixed at proof time). Bounded matching is particularly notable: it allows the prover to demonstrate that any trade within specified bounds satisfies their intent constraints, with the exact size chosen later by the counterparty or a matcher. This enables more flexible order matching by external users, without requiring new proofs for each potential trade size.

The configurations, denoted as **Rings**, are outlined below:

- **Ring 0**: Public Intent, Public Balance
- **Ring 1**: Private Intent, Public Balance
- **Ring 2**: Private Intent, Private Balance, Public Fill
- **Ring 3**: Private Intent, Private Balance, Private Fill

Public Intent, Public Balance (Ring 0)

This configuration does not provide any additional privacy to the traders, and instead prioritizes protocol efficiency. The intents in this configuration are public (they are private until the first fill, when they are registered on-chain for the first time and become public), and the tokens are drawn from Externally Owned Accounts on-demand, with a Permit2 transfer, which is a public transaction.

This configuration is preferred for efficiency, as there is no need for zero-knowledge proofs of validity for the trade components, and for also allowing users who are not members of the Renegade dark pool to transact.

Private Intent, Public Balance (Ring 1)

This privacy configuration is similar to Ring 0, with the main difference being that intents are now private. These intents have been committed to a Merkle tree, and to remain private, zero-knowledge proofs are needed to prove predicates on them for validity, for settlement (normal or bounded) and proof linking (see our previous report) . This incurs an extra computational cost compared to Ring 0.

As in Ring 0, balances are capitalized by EOA transfers, and are thus public. Importantly, even though intents are private, fills in this configuration *remain public*. As a result, an external observer of the chain can learn about the fill sizes but the intent size remains unknown.

Private Intent, Private Balance, Public & Private Fill (Ring 2)

In this configuration, both intents and balances are committed to a Merkle tree and remain private throughout the trade. The configuration of Ring 2 is available only for internal dark pool balances, not external users, and zero-knowledge proofs are now required for both elements upon every fill. Consequently, this carries additional computational costs compared to the previous configurations.

It's important to note, however, that Ring 2 can allow for *both* public fills and private fills. This achieves compatibility of Ring 2 with both Ring 0 and Ring 1, where fills are public, but also with Ring 3, where fills are private.

Private Intent, Private Balance, Private Fill (Ring 3)

This configuration was the default and only configuration in V1. It is maximally private, with both intents and balances being private, and fills also being strictly required to be private. Zero-knowledge proofs are needed for all three elements: for validity of intents and balances, and for the private settlement. Additional proof linking arguments are also needed to link the proofs of validity to the settlement proof. As a result, this configuration is the most computationally expensive one of the four.

Circuits

The V2 circuit system is organized into validity proofs and settlement proofs, connected via proof linking. Validity proofs (`IntentAndBalanceValidity` , `IntentOnlyValidity` , `OutputBalanceValidity`) verify that state elements exist in the darkpool's Merkle tree and satisfy their constraints. Settlement proofs then verify that state updates from a match are consistent with the validated inputs. This two-phase approach, combined with proof linking, allows values to remain hidden in validity proof statements while being selectively revealed during settlement—preventing match details from leaking through calldata analysis.

V2 distinguishes between first-fill and subsequent-fill circuits. When an intent is first used, authorization must bootstrap from the user's signing key through their balance to the intent. The `IntentAndBalanceFirstFillValidity` circuit handles this by verifying a Schnorr signature over the intent commitment using the balance's authority key. Subsequent fills can skip this authorization step since the intent already exists in the Merkle tree with a known commitment. This optimization reduces proving costs for partial fills while maintaining security through the initial authorization chain.

Fee handling is integrated directly into the balance state, with `relayer_fee_balance` and `protocol_fee_balance` fields accumulating fees from settlements. Dedicated fee circuits (`ValidPrivateRelayerFeePayment` , `ValidPrivateProtocolFeePayment`) handle fee redemption, converting accumulated fee balances into notes that can be claimed by relayers and the protocol. The note redemption circuit (`ValidNoteRedemption`) then allows these notes to be converted to withdrawable funds.

Contracts

The V2 contracts implement the on-chain settlement layer using zero-knowledge proofs, specifically the *PlonK* proving system over the BN254 elliptic curve, with state commitments maintained in a Merkle mountain range.

Users interact with the `DarkpoolV2` contract located in `v2/contracts/DarkpoolV2.sol`. The contract exposes two categories of operations that require zero-knowledge proof verification to ensure cryptographic validity:

State Update Operations that modify individual balances and orders:

- `deposit()` / `depositNewBalance()`: Add funds to existing or new balances
- `withdraw()`: Remove funds from balances
- `cancelPrivateOrder()`: Cancel private orders
- `payPublicProtocolFee()` / `payPublicRelayerFee()`: Public fee payments
- `payPrivateProtocolFee()` / `payPrivateRelayerFee()`: Private fee payments
- `redeemNote()`: Redeem fee notes for tokens

Settlement Operations that execute trades between parties:

- `settleMatch()`: Settle internal matches between two Renegade parties
- `settleExternalMatch()`: Settle matches between a Renegade party and external counterparty

Each operation requires submitting a proof bundle that the verifier contract validates before any state changes are committed, ensuring the integrity of all balance updates and preventing invalid state transitions.

The contract follows a modular library-based architecture, with most business logic delegated to specialized libraries in `v2/libraries/`:

- `settlement/*.sol`: Implements settlement logic for four different settlement types:
  - Native-settled public intents (with on-chain ERC20 transfers)
  - Native-settled private intents (with on-chain ERC20 transfers)
  - Renegade-settled private intents (in-circuit settlement, no transfers)
  - Renegade-settled private fills (in-circuit settlement)

- `DarkpoolState.sol`: Manages all protocol state including:
  - Merkle Mountain Range for state commitments
  - Nullifier set for double-spend prevention
  - Public intent tracking for transparent order books
  - Signature nonce tracking for replay protection
  - Token whitelist and per-pair fee overrides
- `TransferLib.sol`: Handles all ERC20 token transfers to/from the Darkpool:
  - Permit2 integration for gasless approvals
  - WETH wrapping/unwrapping for native token support
  - Witness-based deposits that cryptographically bind deposits to specific balance updates
  - Signed withdrawals requiring owner authorization

Beyond cryptographic protections, the contract implements further security through operational controls:

- Upgradeable: Uses the Transparent Proxy Pattern (EIP-1967), allowing the protocol to fix bugs or add features while preserving state and user balances
- Pausable: Emergency stop mechanism that the owner can activate to halt all state-modifying operations during security incidents
- Two-Step Ownership: Prevents accidental ownership transfer by requiring the new owner to accept ownership

Findings

Below are listed the findings found during the engagement. High severity findings can be seen as so-called "priority 0" issues that need fixing (potentially urgently). Medium severity findings are most often serious findings that have less impact (or are harder to exploit) than high-severity

findings. Low severity findings are most often exploitable in contrived scenarios, if at all, but still warrant reflection. Findings marked as informational are general comments that did not fit any of the other criteria.

ID	COMPONENT	NAME	RISK
#00	settlement_lib.rs, valid_private_relayer_fee_payment.rs, valid_private_protocol_fee_payment.rs	Fee balance updates missing amount bitlength constraint	**Medium**
#01	nullifier.rs	Nullifier gadget computes but does not enforce index != 0 constraint	**Medium**
#02	valid_balance_create.rs, intent_and_balance_first_fill.rs, schnorr.rs	User-provided signing authority lacks on-curve and subgroup validation	**Low**
#04	DarkpoolV2.sol, TransferLib.sol	DarkpoolV2 missing receive() function for native ETH withdrawals	**Informational**

ID	COMPONENT	NAME	RISK
#05	DarkpoolV2.sol, MerkleTree.sol, MerkleZeros.sol, VKeys.sol, RenegadeSettledPrivateIntent.sol	Stale and misleading comments	**Informational**
#06	WalletUpdateVKeys.sol, SettlementVKeys.sol, VKeys.sol, ISettlementTypes.sol	Verification key duplication and potential dead code	**Informational**

#00 - Fee balance updates missing amount bitlength constraint

Severity: Medium **Location:** settlement_lib.rs, valid_private_relayer_fee_payment.rs, valid_private_protocol_fee_payment.rs

Description. In V2's `verify_output_balance_share_updates` (`settlement_lib.rs:166-186`), the `relayer_fee_balance` and `protocol_fee_balance` fields are updated by addition but are not constrained to the `2^AMOUNT_BITS` bound specified in the protocol.

```
// settlement_lib.rs — fee balances updated without range check
let new_relayer_fee_balance = cs.add(old_relayer_fee_balance,
relayer_fee)?;
let new_protocol_fee_balance = cs.add(old_protocol_fee_balance,
protocol_fee)?;
```

The main `amount` field is indirectly protected via `verify_out_balance_constraints` (lines 97-112), but the fee balance fields have no such constraint.

V1's equivalent function `validate_no_receive_overflow_singleprover` (`valid_match_settle/single_prover.rs:362-377`) explicitly applied `AmountGadget::constrain_valid_amount` to all three updated fields.

Additionally, V2 removed secondary amount checks that V1 had at fee payment and redemption time. V1's `valid_offline_fee_settlement.rs:165` called `constrain_valid_amount` on the note amount (set to the fee balance), and `valid_fee_redemption.rs` checked amounts during wallet state transitions. V2's equivalent circuits (`valid_private_relayer_fee_payment.rs` , `valid_private_protocol_fee_payment.rs` , `valid_note_redemption.rs`) removed these checks, eliminating defense-in-depth.

Impact. This is a soundness regression that allows fee balances to drift beyond `AMOUNT_BITS` and violate a global invariant that other components assume. While there is no direct fund theft vector, the consequences include:

Accumulated out-of-range fee balances could cause permanent liveness issues. Fee redemption notes may become unredeemable if the contract lacks sufficient balance to honor them, creating a DoS on fee payouts. Other downstream logic that assumes the invariant holds could also be affected.

Recommendation. Add `AmountGadget::constrain_valid_amount` to the fee balance updates in `verify_output_balance_share_updates`, matching V1's behavior. Consider also restoring the secondary checks at fee payment and note redemption time to provide defense-in-depth.

#01 - Nullifier gadget computes but does not enforce index != 0 constraint

Severity: Medium **Location:** nullifier.rs

Description. In `nullifier.rs:30`, the `NullifierGadget::compute_nullifier` function includes a comment stating “The recovery stream index cannot underflow”, followed by a call to `NotEqualGadget::not_equal`:

```
// The recovery stream index cannot underflow
NotEqualGadget::not_equal(element.recovery_stream.index, cs.zero(), cs)?;

// ... later ...
let index_minus_one = cs.sub(element.recovery_stream.index, cs.one())?;
```

However, `not_equal` returns a boolean result that is discarded. Unlike `LessThanGadget::constrain_less_than` (in `comparators.rs:278-286`), which calls `cs.enforce_true(result)` after computing the comparison, this code never enforces the constraint.

As a result, a prover can supply `index=0`, causing the subsequent `index - 1` computation to underflow to `p-1` (where `p` is the field modulus). The circuit will accept this invalid state.

A test confirms the bug:

```
#[test]
fn test_nullifier_gadget_rejects_zero_index() -> Result<()> {
    let mut rng = thread_rng();
    let scalar = Scalar::random(&mut rng);
    let mut elt = create_random_state_wrapper(scalar);
    elt.recovery_stream.index = 0; // Force index to 0

    let mut cs = PlonkCircuit::new_turbo_plonk();
    let elt_var = elt.create_witness(&mut cs);
    let _nullifier_var = NullifierGadget::compute_nullifier(&elt_var, &mut
cs)?;

    // Should be unsatisfiable, but passes due to the bug
    assert!(cs.check_circuit_satisfiability(&[]).is_err(),
        "Circuit should reject index=0 but it doesn't");
```

```
    Ok(())
}
```

Impact. This is a soundness bug: the circuit accepts proofs it should reject. However, the impact is limited because:

1. The `recovery_stream.index` is part of the committed balance state (via `compute_private_commitment`), so a prover cannot arbitrarily change it without breaking the Merkle commitment.
2. The nullifier remains a deterministic function of the committed state. Even with underflow, the same balance produces the same (malformed) nullifier, preventing double-spend.

The bug allows balances with `index=0` to be spent when the system invariant assumes they cannot. This could cause issues if other parts of the system assume `index >= 1`, or if recovery flows depend on valid index values.

Recommendation. Enforce the constraint by calling `cs.enforce_true()` on the result:

```
let not_zero = NotEqualGadget::not_equal(element.recovery_stream.index,
cs.zero(), cs)?;
cs.enforce_true(not_zero)?;
```

#02 - User-provided signing authority lacks on-curve and subgroup validation

Severity: Low **Location:** valid_balance_create.rs, intent_and_balance_first_fill.rs, schnorr.rs

Description. The user-provided `authority` (a BabyJubJub public key) is accepted without on-curve or subgroup validation, both in-circuit and on-chain.

The key is first introduced in `valid_balance_create.rs:149` as part of the witness (`ValidBalanceCreateWitness.balance.authority`) and committed at lines 64-65 without any curve validation:

```
// valid_balance_create.rs:64-65
let commitment = CommitmentGadget::compute_commitment(&new_balance,
&private_share, cs)?;
EqGadget::constrain_eq(&commitment, &statement.balance_commitment, cs)?;
```

The key is later used in `intent_and_balance_first_fill.rs:81-86` for Schnorr signature verification:

```
SchnorrGadget::verify_signature(
    &witness.intent_authorization_signature,
    &original_intent_commitment,
    &witness.balance.authority,
    cs,
)?;
```

The `SchnorrGadget::verify_signature` implementation (in `schnorr.rs:14-28`) passes the public key directly to jellyfish's `SignatureGadget::verify_signature` without any on-curve validation. The Solidity contracts also accept the key as-is without validation.

For BabyJubJub (a twisted Edwards curve with cofactor 8), accepting arbitrary points without validation could cause undefined EC arithmetic in the gadget, potentially weakening signature soundness. Small-subgroup points could also weaken signature security.

Impact. The impact is limited because users choose their own `authority`, and the Permit2 witness binds users to their chosen key. This prevents cross-user attacks—a malicious user providing an invalid key can only make their own signature checks unreliable.

However, the lack of validation violates defense-in-depth principles and could become problematic if the protocol changes how authorities are assigned or derived, or if a user disputes transactions claiming their signatures were forged due to a weak key.

Recommendation. Consider the security/performance trade-off. For in-circuit validation, add an explicit on-curve check (and ideally subgroup validation or cofactor clearing) for the authority when it is first introduced in `valid_balance_create.rs`. This provides defense-in-depth for all downstream Schnorr verifications. If contract-side curve checks are feasible, validate the key at registration time; otherwise document the trust model and limitation explicitly.

#04 - DarkpoolV2 missing receive() function for native ETH withdrawals

Severity: Informational **Location:** DarkpoolV2.sol, TransferLib.sol

Description. V1's `Darkpool.sol` included a `receive() external payable {}` function (line 182) to accept ETH transfers. V2's `DarkpoolV2.sol` removes this function entirely.

In V2, `TransferLib.executeSimpleWithdrawal` (lines 200-204) contains logic for native token withdrawals by unwrapping WETH:

```
if (DarkpoolConstants.isNativeToken(transfer.mint)) {
    wrapper.withdraw(transfer.amount);
    SafeTransferLib.safeTransferETH(transfer.account, transfer.amount);
    return;
}
```

When `wrapper.withdraw()` is called, the WETH contract sends ETH back to `msg.sender`, which in the delegatecall context is the proxy address. Without a `receive()` function, this ETH transfer would fail.

The Renegade team confirmed that native payments for external matches are not yet implemented, and this code path is currently unreachable. However, the logic remains in `TransferLib` as placeholder for future use.

Impact. No immediate impact since native payments are not currently enabled. However, if native ETH withdrawals are enabled in the future without adding a `receive()` function, withdrawals would revert and users would be unable to withdraw funds in native form.

Recommendation. Either remove the native payment logic from TransferLib until the feature is properly implemented, or add `receive() external payable {}` to `DarkpoolV2.sol` now to ensure the code path works when enabled. Keeping dead code that would fail if activated creates a latent bug risk.

#05 - Stale and misleading comments

Severity: Informational **Location:** DarkpoolV2.sol, MerkleTree.sol, MerkleZeros.sol, VKeys.sol, RenegadeSettledPrivateIntent.sol

Description. Several comments in the codebase are outdated, misleading, or incomplete:

- `DarkpoolV2.sol:88-89` : Comment mentions `merkleTree` but actual field is `merkleMountainRange` . Also omits `whitelistedTokens` from storage layout description.
- `DarkpoolV2.sol:79-82` : Comments for `permit2` and `weth` describe V1 scope (“for handling deposits”) but V2 uses these in settlement and fee payment too.
- `MerkleTree.sol:155` : Comment references “depth” but code uses `height` variable.
- `MerkleTree.sol:25` : Event parameter `new_value` describes the sibling’s existing value, not a newly computed value. Name is potentially misleading.
- `MerkleZeros.sol:6` : Comment states `ZERO_VALUE_0` is `keccak256("renegade")` but doesn’t document that higher values are derived by repeated Poseidon hashing: `ZERO_VALUE_i = Poseidon(ZERO_VALUE_{i-1}, ZERO_VALUE_{i-1})` .
- `VKeys.sol:19` : Comment says “(placeholder)” but `ProofLinkingVKeys.sol` is fully implemented with 5 linking key functions. Text appears stale.
- `RenegadeSettledPrivateIntent.sol:56-57, 176-177` : Comment states “balance constraint is implicitly checked by transferring into the darkpool” but this file handles private (Merkle-committed) balances. Appears copy-pasted from `NativeSettledPublicIntent.sol` .

Impact. No security impact. Misleading comments can cause confusion during development and future audits.

Recommendation. Update comments to reflect the current implementation.

#06 - Verification key duplication and potential dead code

Severity: Informational **Location:** WalletUpdateVKeys.sol, SettlementVKeys.sol, VKeys.sol, ISettlementTypes.sol

Description. The verification key contracts contain duplicated functions and potential dead code:

- `WalletUpdateVKeys.sol:9`: Documented as "Verification keys for wallet update operations" but includes 7 settlement/validity key functions (lines 65-105): `intentOnlyFirstFillValidityKeys`, `intentOnlyValidityKeys`, `intentAndBalanceFirstFillValidityKeys`, `intentAndBalanceValidityKeys`, `intentOnlyPublicSettlementKeys`, `intentAndBalancePublicSettlementKeys`, `intentAndBalancePrivateSettlementKeys`.
- These 7 functions are duplicated in `SettlementVKeys.sol`. The main `VKeys.sol` only delegates to `SETTLEMENT_VKEYS` for these keys (lines 90-136), making the `WalletUpdateVKeys` copies unreachable through the standard interface.
- The duplication is incomplete: `SettlementVKeys.sol` includes `intentOnlyBoundedSettlementKeys()` and `intentAndBalanceBoundedSettlementKeys()`, but `WalletUpdateVKeys` does not have these. If duplication was intentional, it appears incomplete.
- `ISettlementTypes.sol`: Imports `PrivateIntentPublicBalanceBoundedFirstFillBundle` and `PrivateIntentPublicBalanceBoundedBundle` twice (back-to-back). Redundant but harmless.

Impact. No security impact. Duplicated code increases maintenance burden and could lead to inconsistencies if one copy is updated without the other.

Recommendation. Remove the duplicated settlement key functions from `WalletUpdateVKeys.sol` if not needed, or document the intended deployment pattern if duplication is deliberate. Clean up redundant imports

in `ISettlementTypes.sol`.